

CHALLENGE WTC 2025 · FIAP · ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

WTC Chat App

Plataforma de comunicação CRM — app iOS nativo + backend Java robusto, integrados via APIs REST reais e mensageria em tempo real.

SPRINT 2 Backend real · sem mocks

Documentação técnica completa — arquitetura · modelo de dados · endpoints · execução · testes

Arthur Cavalcanti Granja · RM 560650 | Entrega: 19/05 · 23h00 | 2TDS

INTEGRANTES

Equipe

Arthur Cavalcanti Granja

RM 560650

Análise e Desenvolvimento de Sistemas — 2TDS · FIAP
Challenge WTC 2025 · Sprint 2 — Backend + Integração · Equipe SOLO

Repositório: github.com/artcgranja/Challenge-WTC

Visão Geral

PROBLEMA

- Comunicação fragmentada entre empresa e clientes
- Falta de personalização nas mensagens de marketing
- Notificações sem interatividade
- Dificuldade em segmentar o público-alvo

SOLUÇÃO

- Mensagens segmentadas 1:1 e por grupo/tag
- Push + pop-up in-app, histórico de chat
- Botões de ação e deeplinks interativos
- CRM no app (operador) + tempo real (WebSocket)

WTC Business Club São Paulo — operadores/marketing disparam promoções, campanhas, banners e eventos diretamente para clientes ou segmentos.

DIAGRAMA

Arquitetura do Backend



Camadas: Controller (REST) → Service (regra de negócio) → Repository (Spring Data) → MongoDB. Em paralelo: cria Notification + broadcast WebSocket; AuditAspect (AOP) registra toda escrita.

Fluxo de Envio de uma Mensagem

1	Operador autentica	POST /api/auth/login → recebe JWT (access 24h). Header Authorization: Bearer <token> em todas as chamadas seguintes.
2	Envia a mensagem	POST /api/messages com type CHAT (1:1, recipient_id) ou CAMPAIGN (segment_tags[]). JwtAuthFilter valida o token e injeta o senderId.
3	Persiste no MongoDB	MessageService grava o documento na collection messages com status SENT e createdAt.
4	Notifica destinatários	Cria Notification por destinatário (1:1 = recipient; segmento = todos os users cujas tags batem).
5	Broadcast em tempo real	WebSocketService publica em /topic/messages/{userId} via STOMP — app recebe sem polling.
6	Auditoria automática	AuditAspect (@AfterReturning) grava em audit_logs: userId, action, resource, resourceId, ipAddress, timestamp.

Stack & Segurança

iOS	Swift 5.9 · SwiftUI · Combine · MVVM · iOS 15+ · sem dependências externas (URLSession + STOMP manual)
Backend	Java 17 · Spring Boot 3.3 · Spring Security · Spring Data MongoDB · Spring WebSocket · Spring AOP · Lombok · JWT
Banco	MongoDB 6.0+ — NoSQL orientado a documentos, 7 collections, IDs UUID string
Auth	JWT (access 24h / refresh 7d) · BCrypt · roles OPERATOR / CLIENT no claim
Realtime	WebSocket STOMP — /topic/messages/{userId} · /topic/notifications/{userId}
Governança	Auditoria automática via AOP (@AfterReturning) em toda operação de escrita (POST/PUT/DELETE)

Modelo de Dados – 7 Collections

users

operadores + clientes – auth, perfil, role, tags[], status

customers

registro CRM – userId, tags[], score 0-100, status, notes[], segmentIds[]

segments

name, description, tags[], createdBy

messages

type, senderId, recipientId, segmentTags[], content{}, status, readAt, starred

campaigns

name, segmentId, content{}, deeplink, status, sentBy, messageCount

notifications

userId, title, body, type, read, messageId

audit_logs

userId, action, resource, resourceId, details, ipAddress, timestamp

Relações por referência (sem joins): *customers.userId* → *users.id* · *campaigns.segmentId* → *segments.id* · *notifications.messageId* → *messages.id*

Documentos: users & customers

users

```
{
  "id": "uuid",           // _id
  "email": "joao@test.com", // único*
  "password": "$2a$ (BCrypt)",
  "fullName": "João Silva",
  "phone": "(11) 99999-1111",
  "avatarUrl": null,
  "role": "CLIENT",      // OPERATOR|CLIENT
  "tags": ["vip", "ativo"],
  "status": "active",
  "createdAt": "2026-05-19T...",
  "updatedAt": "2026-05-19T..."
}
```

customers

```
{
  "id": "uuid",
  "userId": "uuid (→ users.id)*",
  "tags": ["vip", "ativo"],
  "score": 85,           // 0-100
  "status": "ACTIVE",    // ACTIVE
                          // INACTIVE
                          // PENDING
  "notes": [
    { "text": "Cliente VIP",
      "createdBy": "uuid",
      "createdAt": "..." }
  ],
  "segmentIds": ["uuid"],
  "createdAt": "...", "updatedAt": "..."
}
```

* campo indexado · id = UUID string (compatível com tipo UUID do iOS) · senhas em BCrypt, nunca retornadas pela API

Documentos: segments & messages

segments

```
{
  "id": "uuid",
  "name": "VIP",
  "description":
    "Clientes VIP do WTC",
  "tags": ["vip"],
  "createdBy": "uuid",
  "createdAt": "...",
  "updatedAt": "..."
}
```

messages (content embute buttons[])

```
{
  "id": "uuid",
  "type": "CAMPAIGN",           // CHAT|CAMPAIGN
  "senderId": "uuid",
  "recipientId": "uuid*",      // 1:1 (CHAT)
  "segmentTags": ["vip"],      // broadcast
  "content": {
    "title": "Black Friday WTC 2026",
    "body": "Condições exclusivas...",
    "imageUrl": "https://.../img.jpg",
    "buttons": [
      { "label": "Inscrever-se",
        "action": "https://wtc.com/bf" }
    ]
  },
  "status": "SENT",           // SENT|DELIVERED|READ|FAILED
  "readAt": null, "starred": false,
  "createdAt": "..."
}
```

Documentos: campaigns · notifications · audit_logs

campaigns

```
{
  "id": "uuid",
  "name": "Black Friday",
  "segmentId": "uuid",
  "content": { ... },
  "deeplink":
    "deeplink://promo",
  "status": "SENT",
    // DRAFT|SENT
  "sentAt": "...",
  "sentBy": "uuid",
  "messageCount": 2,
  "createdAt": "..."
}
```

notifications

```
{
  "id": "uuid",
  "userId": "uuid*",
  "title": "Nova msg",
  "body": "...",
  "type": "MESSAGE",
    // MESSAGE
    // CAMPAIGN
    // SYSTEM
  "read": false,
  "messageId": "uuid",
  "createdAt": "..."
}
```

audit_logs

```
{
  "id": "uuid",
  "userId": "uuid",
  "action": "CREATE",
  "resource":
    "messages",
  "resourceId":
    "uuid",
  "details": "...",
  "ipAddress":
    "127.0.0.1",
  "timestamp": "..."
}
```

Segmentação Server-Side (Inbox)

A inbox de um cliente combina mensagens diretas (1:1) e campanhas cujas tags de segmento batem com as tags do usuário — uma única query MongoDB com \$or, sem joins:

MessageRepository.findInbox(userId, userTags, sort)

```
@Query("{ '$or': [ { 'recipientId': ?0 },  
                  { 'segmentTags': { '$in': ?1 } } ] }")
```

Mensagem 1:1

recipientId = userId → entra na inbox do destinatário direto.

Campanha por tag

segmentTags $\cap$ userTags $\neq \emptyset$ → entra para todos os clientes do segmento.

Vantagem

Filtro no servidor (sem RLS, sem lógica no app);
GET /api/inbox/{id} ordenado por createdAt desc.

Mapa de Endpoints

Grupo	Endpoints	Acesso
Auth	POST /register · /login · /refresh	público
Customers	GET/POST /customers · GET/PUT /{id} · /{id}/timeline · POST /{id}/notes	OPERATOR
Segments	GET/POST /segments · GET/PUT/DELETE /{id}	OPERATOR
Messages	POST /messages · GET /{id} · /inbox/{id} · /sent/{id} · PUT /{id}/read · /star	autenticado
Campaigns	GET/POST /campaigns · POST /{id}/send	OPERATOR
Notifications	GET /notifications · PUT /{id}/read · /read-all	autenticado
Audit	GET /audit-logs?resource&userId	OPERATOR
WebSocket	ws://host:8080/ws → /topic/messages/{id} · /topic/notifications/{id}	STOMP

Prefixo base: /api · todas as rotas (exceto auth) exigem header Authorization: Bearer <jwt>

Auth • público

POST /api/auth/login público

POST /api/auth/register público

POST /api/auth/refresh público

register exige email, password, fullName; role/tags/phone opcionais.
login retorna 200; register 201; refresh troca refresh_token por novo par.

REQUEST POST /login

```
{
  "email":
    "admin@wtc.com",
  "password":
    "admin123"
}
```

RESPONSE 200 OK

```
{
  "token": "eyJ...",
  "refresh_token":
    "eyJ...",
  "user_id": "uuid",
  "email": "...",
  "full_name": "...",
  "role": "OPERATOR",
  "tags": [],
  "status": "active"
}
```

CRM • Customers • OPERATOR

GET	/api/customers	OPERATOR
POST	/api/customers	OPERATOR
GET	/api/customers/{id}	OPERATOR
PUT	/api/customers/{id}	OPERATOR
GET	/api/customers/{id}/timeline	OPERATOR
POST	/api/customers/{id}/notes	OPERATOR

GET aceita filtros: ?tag= ?status= ?minScore=. Lista retorna customer enriquecido com dados do user (full_name, email, phone). timeline = customer + mensagens + notas (perfil 360°).

REQUEST POST

```
{
  "userId":
    "uuid",
  "tags":
    ["vip"],
  "score": 85,
  "status":
    "ACTIVE"
}
```

POST /api/customers/{id}/notes

```
{
  "text":
    "Ligar amanhã"
}
```

RESPONSE 201/200

```
{
  "id": "uuid",
  "user_id": "uuid",
  "full_name":
    "João Silva",
  "email": "...",
  "tags": ["vip"],
  "score": 85,
  "status": "ACTIVE",
  "notes": [...],
  "segment_ids": []
}
```

Segments & Campaigns • OPERATOR

GET/POST	/api/segments	OPERATOR
GET	/api/segments/{id}	OPERATOR
PUT/DELETE	/api/segments/{id}	OPERATOR
GET/POST	/api/campaigns	OPERATOR
POST	/api/campaigns/{id}/send	OPERATOR

POST /campaigns cria DRAFT. POST /{id}/send dispara para o segmento: gera uma mensagem CAMPAIGN por destinatário, marca status SENT, sentAt e messageCount.

REQUEST campaign

```
{
  "name":
    "Black Friday",
  "segmentId":
    "uuid",
  "content": {
    "title": "...",
    "body": "..."
  },
  "deeplink":
    "deeplink://x"
}
```

RESPONSE 201

```
{
  "id": "uuid",
  "name": "...",
  "segmentId":
    "uuid",
  "status":
    "DRAFT",
  "sentBy":
    "uuid",
  "messageCount": 0
}
```

Messages • autenticado

POST	/api/messages	autenticado
GET	/messages/{id}	autenticado
GET	/inbox/{id}	autenticado
GET	/sent/{id}	autenticado
PUT	{id}/read	autenticado
PUT	{id}/star	autenticado

REQUEST POST /messages

```
{
  "type": "CHAT",
  "recipient_id":
    "uuid-cliente",
  "content": {
    "title": "Olá!",
    "body": "Mensagem...",
    "image_url": "https://",
    "buttons": [
      { "label": "Ver",
        "action":
          "deeplink://profile" }
    ]
  }
}
```

RESPONSE 201

```
{
  "id": "uuid",
  "type": "CHAT",
  "sender_id": "uuid",
  "recipient_id":
    "uuid",
  "content": { ... },
  "status": "SENT",
  "starred": false,
  "created_at": "..."
}
```

SENT→DELIVERED
→READ →FAILED

Notifications & Audit

GET	/api/notifications	autenticado
PUT	/notifications/{id}/read	autenticado
PUT	/notifications/read-all	autenticado
GET	/api/audit-logs	OPERATOR

notifications: userId vem do JWT (não da URL). audit-logs aceita ?resource= e ?userId=. PUT read / read-all retornam 200 sem corpo.

RESP notifications

```
[
  {
    "id": "uuid",
    "userId": "uuid",
    "title": "Nova msg",
    "body": "...",
    "type": "MESSAGE",
    "read": false,
    "messageId":
      "uuid"
  }
]
```

RESP audit-logs

```
[
  {
    "id": "uuid",
    "userId": "uuid",
    "action": "CREATE",
    "resource":
      "messages",
    "resourceId":
      "uuid",
    "ipAddress":
      "127.0.0.1",
    "timestamp": "..."
  }
]
```

Tempo Real · WebSocket / STOMP

Endpoint & Tópicos

```
ws://localhost:8080/ws (STOMP)
/topic/messages/{userId}
/topic/notifications/{userId}
```

Quando uma msg é enviada via REST

1. Salva no MongoDB
2. Cria Notification por destinatário
3. Broadcast STOMP → app sem polling

Por que é um diferencial

Nova mensagem aparece instantaneamente no app — sem refresh, sem polling. O cliente iOS implementa um STOMP client manual sobre URLSessionWebSocketTask (zero dependências externas), atualizando badge e lista em tempo real.

Auditoria Automática (Spring AOP)

Um único AuditAspect intercepta toda operação de escrita com @AfterReturning — zero código manual nos controllers/services. Cada CREATE/UPDATE/DELETE vira um registro imutável em audit_logs.

AuditAspect

```
@AfterReturning("POST|PUT|DELETE  
em controllers")  
→ grava AuditLog
```

userId	quem executou (do JWT)
action	CREATE · UPDATE · DELETE
resource	messages · customers · ...
resourceId	id do documento afetado
ipAddress	origem da requisição
timestamp	data/hora (Instant)

Consulta: GET /api/audit-logs?resource=messages&userId=<uuid> (OPERATOR) — trilha completa para compliance.

Diferenciais

Tempo real

WebSocket/STOMP: nova mensagem aparece na hora, sem polling.

Governança (AOP)

Auditoria automática de toda escrita — zero código manual.

Multi-role

Roteamento por papel: CLIENT (inbox) vs OPERATOR (CRM completo).

Botões interativos

deeplink:// (interno), https:// (externo), copy: (clipboard).

Status de mensagem

Ciclo SENT → DELIVERED → READ → FAILED no documento.

Zero deps no iOS

App nativo sem libs; STOMP client implementado manualmente.

Rodar o Backend & Popular o Banco

Passo a passo (macOS)

```
# 1. Pré-requisitos
brew install openjdk@17 \
  mongodb-community xcodegen
brew services start mongodb-community

# 2. Subir o backend
cd backend
JAVA_HOME=$(brew --prefix openjdk@17) \
  ./mvnw spring-boot:run
# → http://localhost:8080

# 3. Seed automático (1ª execução)
#   DataLoader roda se o banco
#   estiver vazio. Para re-popular:
mongosh wtc_chatapp --eval \
  "db.dropDatabase()"
# e reinicie o backend
```

Seed inicial (DataLoader)

- 5 usuários (2 OPERATOR · 3 CLIENT)
- 3 customers (CRM, score 60–92)
- 3 segmentos (VIP · Ativos · Beta)
- 6 mensagens (1 CHAT · 5 CAMPAIGN)
- 2 campanhas (status SENT)
- Idempotente: só roda se count() = 0

EXECUÇÃO

Rodar o App iOS

Passo a passo

```
# Pré-requisitos: macOS,  
# Xcode 15+, XcodeGen  
  
cd Challenge-WTC  
xcodegen generate  
open WTCChatApp.xcodeproj  
# Cmd+R → simulador iPhone 15+  
# Backend rodando em :8080
```

Integração

- Base URL em Utils/Constants.swift (localhost:8080)
- Jackson snake_case ↔ CodingKeys do iOS
- JWT salvo após login; enviado em toda chamada
- WebSocket conecta após login → realtime

Entregável: app.zip + WTCChatApp.ipa (archive unsigned, CODE_SIGNING_ALLOWED=NO). Para rodar em device físico, abra no Xcode e assine com sua conta de desenvolvedor Apple.

Como Testar — Credenciais & Cenários

Email	Senha	Tipo	Tags
admin@wtc.com	admin123	OPERATOR	—
operador@wtc.com	oper123	OPERATOR	—
joao@test.com	test123	CLIENT	vip, ativo
maria@test.com	test123	CLIENT	ativo
pedro@test.com	test123	CLIENT	vip, beta, ativo

Cenários

- CLIENT (joao): inbox VIP+ativo, ler/favoritar, notificações, perfil
- OPERATOR (admin): CRM, filtros, notas, enviar 1:1/segmento, campanhas
- Realtime: envie por admin → chega na hora no app do joao

Teste rápido via curl

```
curl -s localhost:8080/api/auth/login -H 'Content-Type: application/json' \
-d '{"email":"admin@wtc.com","password":"admin123"}' # → copie o token e use em Authorization: Bearer <token>
```

Resumo da Entrega

backend.zip

código-fonte por camadas + README de execução

app.zip + WTCChatApp.ipa

app iOS integrado ao backend (APIs REST reais)

Apresentação .pdf / .pptx

nomes/RM · arquitetura · endpoints · modelo NoSQL · execução